

Operating System Fundamentals

Module 1 – Windows Fundamentals

Windows OS Fundamentals

Windows is an Operating System developed by Microsoft. It is a graphical user interface (GUI) based system that allows users to interact with their computers and manage various tasks. It runs applications and provides a user-friendly interface that enables users to interact directly with the system to perform tasks.

Windows OS offers distinctive features such as:

- Users are allowed to run the software.
- Users interact with the OS through visual elements, such as icons, windows, menus, and a mouse-driven pointer.
- Users can view or store their files.
- Users can create, delete, rename, and move files using the GUI or the CLI.

Key Characteristics of Windows OS

Graphical User Interface (GUI): The primary user interface is graphical, i.e., instead of typing the commands you manipulate various graphical objects (such as icons) with a pointing device.

File Management: File management refers to the process of organizing and controlling data files stored on the cloud or a device, such as a computer or smartphone.

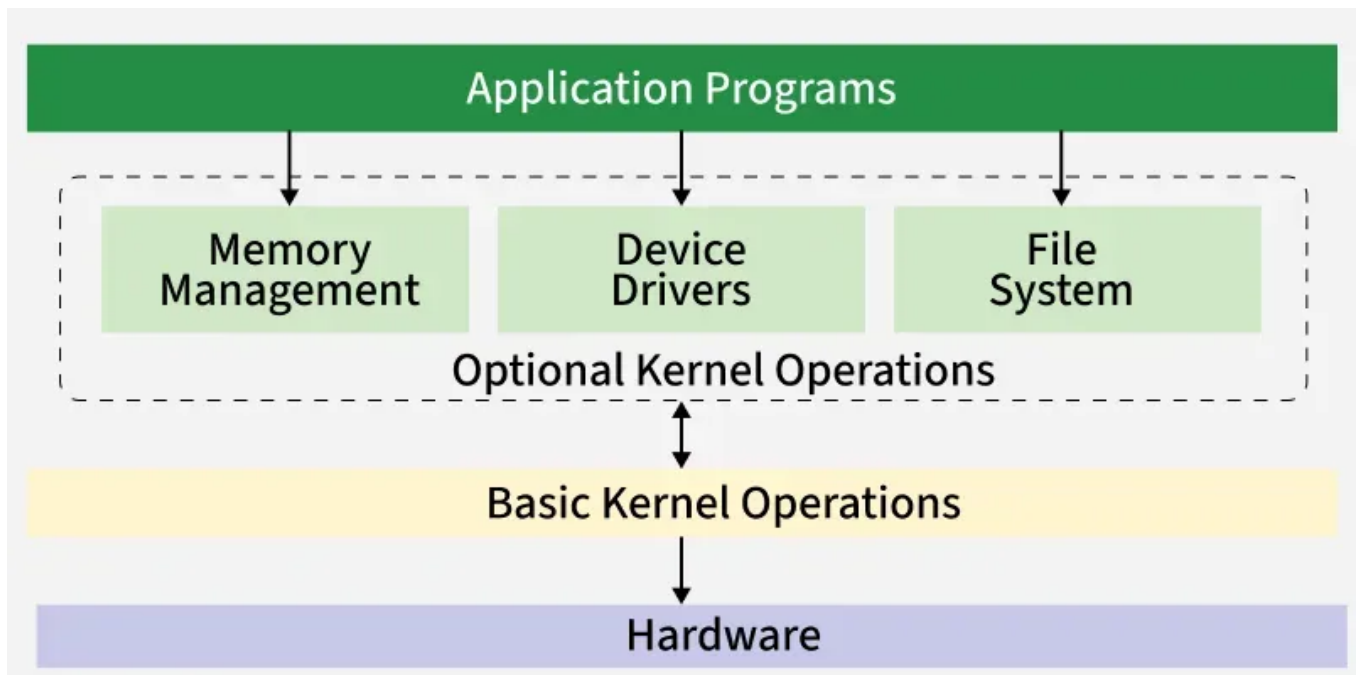
Task Manager: It is built-in system utility in Microsoft Windows that provides detailed information about the performance of your computer and the applications, processes, and services currently running.

Control Panel: Allows users to view and change system settings. It serves as a centralized interface where you can manage the hardware, software, user accounts, and various system configurations.

Built-in security tools: Built-in security tools designed to provide protection at various levels—ranging from antivirus to encryption and network safety. These tools work together to keep your system safe from both internal and external threats.

Virtual desktops: It refers to a logical extension of a physical desktop interface that allows users to create and manage multiple desktop environments on a single computer.

Architecture of Windows OS



Application Programs: An Application Program is a type of computer software designed to perform specific tasks for users. Application Program help users accomplish real-world activities such as creating documents, playing media, browsing the internet, or managing data. Application programs can be installed on various devices, including computers, smartphones, and tablets. They may be bundled with the device or installed separately

Memory Management: Memory Management is a hardware component that stores data, instructions and information temporarily or permanently for processing. It consists of an array of bytes or words, each has a unique address. Memory management in the Windows Operating System is the process through which the OS handles, allocates, and frees physical and virtual memory to ensure efficient performance and multitasking. Windows uses a virtual memory model, which gives applications the illusion of having a large, continuous block of memory, even if the actual physical memory (RAM) is limited.

Device drivers: Device drivers are specialized software programs that act as a bridge between a computer's operating system and its hardware components. The main purpose of device drivers is to provide abstraction by acting as a translator between a hardware device and the applications or Operating System that use it. Programmers can write higher-level application code independently of whatever specific hardware the end-user is using.

File System: A File System is defined as a medium used for saving and managing data in the computer system. The data stored in the computer system is completely in digital format, although there can be various types of files that help us to store the data. A file system is an essential component of any operating system. It ensures

data integrity, security, and usability by managing how files are stored and accessed. Understanding the file system helps users and developers work more effectively with data, system resources, and storage media.

Kernel: A kernel is the core part of an operating system. It acts as a bridge between software applications and the hardware of a computer. The kernel manages system resources, such as the CPU, memory, and devices, ensuring everything works together smoothly and efficiently. It acts as the main interface between the computer's hardware and the software applications that run on the system. The kernel is responsible for managing system resources, such as the CPU, memory, input/output devices, and system calls.

Hardware: Hardware refers to the physical components of a computer system or any electronic device. These are the tangible parts you can touch and see — such as the keyboard, mouse, monitor, CPU, hard drive, and even internal circuits. Hardware is typically directed by the software to execute any command or instructions. A combination of hardware and software forms a usable computing system, although other systems exist with only hardware.

For GRC, you do not need to become a Windows kernel developer. You need to understand **where security controls operate and what evidence Windows can provide**.

Windows architecture is relevant when assessing:

- Endpoint security
- Access control
- Privilege management
- Malware protection
- Patch management
- System hardening
- Logging and monitoring
- Administrative controls

Understanding the difference between user and privileged operations also helps explain why **least privilege** is important.

Boot Process

The Windows boot process is everything that happens from turning on the computer until Windows is ready for the user. First, the computer's firmware, normally UEFI, starts and checks the hardware. It then starts the Windows Boot Manager, which begins loading Windows. Windows loads the necessary operating-system components, kernel, drivers, and services, and finally presents the login screen and user environment.

A simple way to remember it is:

Power On → UEFI → Boot Manager → Windows → Kernel/Drivers → Services → Login

Types of Booting

There are two types of booting depending on the number of operating systems installed on the machine/computer, i.e.

1. Cold or Hard Booting

Cold booting refers to the state in which a computer is switched on after being completely powered off. During this process, the system performs a full power-on self-test (POST), initializes hardware devices, and loads the operating system from storage into RAM.

Cold booting occurs when a computer is powered on from a fully switched-off state.

It is also known as performing a cold boot or fresh startup.

The system carries out POST, initializes hardware, and loads the OS into RAM.

Cold Booting Example,

Pressing the power button on a computer that is completely turned off.

2. Soft or Warm Booting

Warm booting, also known as a soft boot or restart, reboots a computer system without completely shutting it down and is usually triggered through an OS restart command or a key combination. It skips some hardware initialization steps since the components are already powered and initialized, offering a quicker restart compared to a full cold boot.

Warm booting (soft boot/restart) reboots the system without turning it off entirely.

It is initiated through an operating system restart command or a specific key combination.

It skips certain hardware initialization steps, making it faster than a cold boot while still essential for system operation.

Warm Booting Example,

Clicking Restart from the operating system menu (e.g., Windows → Start → Restart) without powering off the computer.

NTFS

NTFS stands for New Technology File System. It is the main file system Windows uses to organize and store files and folders. One of its most important security

features is the ability to control who can access a file or folder and what they can do with it.

For example, you can allow one user to read a file, another user to modify it, and completely block another user from accessing it. These permissions are controlled through Access Control Lists (ACLs).

The simple idea is:

File/Folder → Who can access it? → What can they do?

NTFS permissions are directly connected to data protection, access control, least privilege, and confidentiality. Sensitive business information should only be accessible to people who have a legitimate business requirement.

For GRC, you may review whether permissions on sensitive folders are properly assigned, approved, and periodically reviewed.

Common NTFS permissions include:

Read → View the file/folder.

Write → Create or change content.

Modify → Read, change, and generally delete content.

Full Control → Extensive control, including changing permissions.

ACL → Defines who has which permissions.

Inheritance → Permissions can automatically flow from a parent folder to child folders/files.

Follow least privilege.

Sensitive folders require stronger access restrictions.

Example

A company stores employee salary information in a payroll folder. Only authorized Payroll employees receive the required NTFS permissions. Other employees cannot open the folder, preventing unauthorized access to sensitive information.

Registry

The Windows Registry is a centralized database that stores important configuration information used by Windows, applications, users, and hardware. Instead of keeping every setting in separate files, Windows stores many system and application settings in the Registry.

The Registry is organized into major sections called hives. Important ones include HKEY_LOCAL_MACHINE (HKLM), which contains system-wide settings, and HKEY_CURRENT_USER (HKCU), which contains settings for the currently logged-in user.

Because the Registry controls many important Windows settings, changing the wrong value can break applications, weaken security, or even affect system operation.

The Registry is relevant to GRC because it contains configuration information that can affect security, system hardening, applications, authentication, and endpoint behavior. It can also provide useful evidence during security investigations.

It contains all the information of the software and hardware installed on the system. Users can access and configure the settings of Registries by launching the Registry Editor tool of Windows OS.

The registries can be considered as the DNA of the Windows devices.

Event Viewer

Event Viewer is a Windows tool used to view and analyze system and security event logs. Windows records many events, such as successful and failed logins, application errors, system changes, service activity, and security-related events.

Event Viewer organizes these events into different logs, including Application, Security, and System logs.

Each event can contain information such as the event ID, timestamp, source, severity, and description. This information helps administrators and security teams understand what happened on a system.

Event logs provide valuable evidence for security monitoring, incident investigation, troubleshooting, auditing, and compliance.

For GRC, the key concern is whether important events are being logged, protected, retained, monitored, and reviewed according to organizational requirements.

Application Log

→ Records events related to applications.

Security Log

→ Contains security-related events such as authentication and access activity.

System Log

→ Records events related to Windows system components and services.

Important information includes:

- Event ID

Basic Foundation Notes
GRC Roadmap

- Timestamp
- Event source
- Severity
- User/account involved
- Description

Important security events

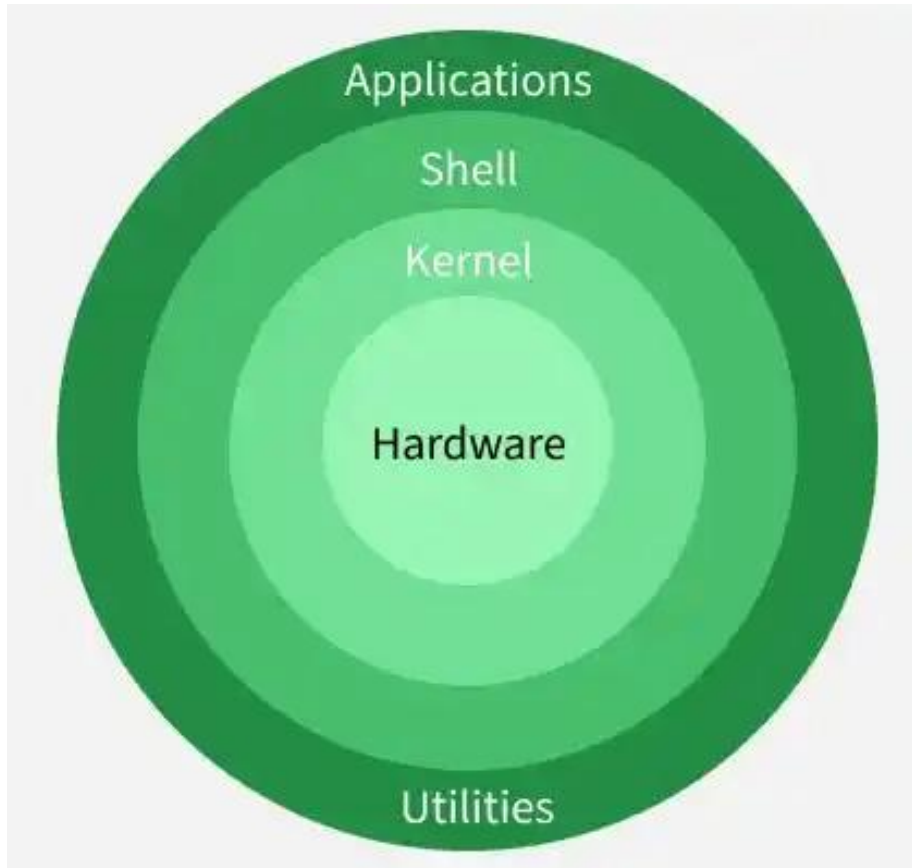
- Successful authentication
- Failed authentication
- Account changes
- Privilege-related activity
- Security policy changes
- Service changes

Logs should ideally be protected from unauthorized modification and forwarded to centralized monitoring/SIEM infrastructure where appropriate.

Module 2 – Linux Fundamentals

Linux Architecture

The Linux operating system architecture defines how different components of the system interact with each other to manage hardware resources, run applications, and provide a stable and secure computing environment. Linux follows a layered architecture, where each layer has a specific role and responsibility.



The main components of Linux operating system are:

- Application
- Shell
- Kernel
- Hardware
- Utilities

Each layer communicates with the one below it, creating a structured and efficient operating system design:

Kernel

Kernel is the core component of the Linux operating system that sits between the hardware and user space, managing system resources and ensuring smooth communication between software and hardware. It controls how processes are executed, scheduled, and isolated to maintain system stability and security.

The kernel is responsible for:

- **Memory management** – Allocates and manages system memory efficiently
- **Process management** – Schedules processes and controls execution using queues
- **Resource allocation** – Distributes CPU, memory, and I/O resources among processes
- **Device management** – Controls hardware devices through device drivers
- **Application interaction** – Acts as a bridge between applications and hardware
- **Security** – Enforces access control and system-level security mechanisms

Shell

The shell is a software interface to the kernel. It takes commands from the user and interprets them. The shell transmits these commands to the kernel, which then performs the requested operations. Users can just enter the command and using the kernel's function that specific task is performed accordingly.

Hardware Layer

The hardware layer is the lowest level of the Linux operating system and forms the foundation on which all other components operate. It consists of physical devices and low-level controls that allow the system to function efficiently and reliably.

- Includes physical components such as CPU, memory, storage, and I/O devices
- Works with device drivers to enable hardware communication
- Supports memory access, CPU control, and I/O operations
- Ensures stable and efficient interaction between hardware and the operating system

System utilities

System utilities are command-line tools that help users and administrators manage, configure, and monitor the Linux system. They simplify system administration by providing ready-to-use commands for common tasks.

- Perform file and directory management operations
- Monitor system performance and resource usage
- Manage users, groups, and permissions
- Configure and troubleshoot network settings

Linux is heavily used in **servers, cloud infrastructure, containers, network devices, security tools, and enterprise applications**. As a GRC professional, you need enough Linux understanding to interpret security configurations, audit evidence, vulnerability reports, access permissions, logs, and hardening requirements.

It also helps you understand where controls are implemented—for example, permissions at the file-system level, user privileges, services, and system logging.

File System

The Linux file system is the structure Linux uses to organize files, directories, devices, configuration information, and other system resources. Unlike Windows, which commonly uses drive letters such as C:\, Linux presents the system as a single hierarchical structure beginning at the root directory /.

Everything is organized under this root. Directories such as /etc, /var, /home, /tmp, and /usr have specific purposes. Linux also follows the principle that many things are represented as files, including certain devices and system interfaces.

Understanding the Linux file system is essential because security configuration, application data, logs, credentials, and system settings are stored in specific locations.

For GRC, the Linux file system matters because important security evidence and configuration information often exists inside it.

You may encounter:

- Configuration files
- Application data
- Authentication information
- System logs
- Security settings
- Backup files
- Temporary files

Incorrect permissions on sensitive files can expose credentials, configuration information, or business data.

/

→ Root of the entire file system.

/home

→ User home directories.

/etc

→ System and application configuration files.

/var

→ Variable data such as logs and application data.

/tmp

→ Temporary files.

/usr

→ Many user applications, libraries, and system utilities.

/bin

→ Essential user commands on systems using this traditional layout.

/sbin

→ System administration commands.

/root

→ Home directory of the root administrator.

/dev

→ Device-related files/interfaces.

/proc

→ Virtual file system exposing information about processes and kernel/system state.

The exact directory structure can vary depending on the Linux distribution and configuration.

Example

During an audit, you need to verify whether Linux authentication configuration is appropriately protected. You may examine relevant configuration files under `/etc` and verify that ordinary users cannot modify them.

Permissions

Linux permissions determine what a user or group can do with a file or directory. The three fundamental permissions are Read (r), Write (w), and Execute (x).

These permissions are assigned separately to three categories:

Owner → Group → Others

For example:

`rxwxr-x---`

means:

- Owner → `rxw` → read, write, execute
- Group → `r-x` → read and execute
- Others → `---` → no access

For files, execute generally means the ability to run the file as a program. For directories, execute has a different meaning: it allows a user to enter/traverse the directory and access items when other permissions allow it.

Linux permissions are a major **access-control mechanism**. Incorrect permissions can expose sensitive information or allow unauthorized users to modify critical system files.

For GRC, permissions are relevant to:

- Least privilege
- Data protection
- Access reviews
- System hardening
- Compliance evidence
- Unauthorized modification prevention

Ownership

Linux ownership defines **which user and which group a file or directory belongs to**. Every file normally has an **owner** and an associated **group**. Linux then uses ownership together with permissions to decide who can read, modify, or execute that resource.

Think of it as:

File → Owner + Group → Permissions

For example, a file might belong to user appuser and group developers. The owner, members of the group, and everyone else can then receive different permissions.

Ownership is separate from permissions, but the two work together. A user who owns a file can potentially change its permissions, depending on the system's controls.

Ownership is important for **access control and least privilege**. Incorrect ownership can allow unauthorized users or applications to modify sensitive files.

For GRC, ownership is also useful for establishing **accountability**. Sensitive files should have an identifiable responsible owner and should not be arbitrarily owned by highly privileged accounts.

- ☐ Every file normally has an **owner**.
- ☐ Every file normally has an associated **group**.
- ☐ Ownership and permissions work together.
- ☐ **User ownership** identifies the owning account.
- ☐ **Group ownership** identifies the associated group.
- ☐ Ownership can be changed by authorized administrators.

- ☐ Sensitive files should have appropriate ownership.
- ☐ Avoid unnecessary ownership by root.
- ☐ Ownership should support least privilege and accountability.

Shell

A Linux **shell is a command-line interface that allows a user to interact with the operating system**. Instead of clicking graphical menus, users type commands into the shell to perform tasks such as creating files, checking processes, managing permissions, viewing logs, and configuring systems.

Common shells include **Bash, Zsh, and Fish**. Bash is particularly common in enterprise Linux environments.

The shell interprets commands and asks the operating system to perform the requested actions. Because commands can perform powerful administrative operations, shell access must be appropriately controlled.

For GRC, shell access is important because it provides direct interaction with systems and can allow powerful administrative actions.

You should understand:

- Who has shell access?
- Is privileged access restricted?
- Are administrative actions logged?
- Are shared accounts avoided?
- Are commands or sessions monitored where required?
- Are users given only the access they need?

Package Management

Package management is the process of **installing, updating, removing, and maintaining software packages on a Linux system**. A package generally contains software and the information required to install it correctly.

Package managers are important because they help administrators install software and dependencies in a controlled and repeatable way.

Outdated software can contain known vulnerabilities. Therefore, package management is directly connected to vulnerability management, patch management, system hardening, and compliance.

For GRC, you need to know whether organizations:

- Maintain supported software versions

- Apply security updates
- Track vulnerabilities
- Control software installation
- Remove unnecessary packages
- Maintain evidence of patching

Module 3 – Enterprise Administration

Active Directory

Active Directory (AD) is Microsoft's centralized system for managing users, computers, groups, authentication, and access in a Windows enterprise environment. Instead of configuring every computer and user separately, an organization can manage identities and security centrally through Active Directory.

AD stores information about objects such as users, computers, groups, and organizational units. It also works closely with Domain Controllers, which provide authentication and directory services.

The important idea is:

Central Directory → Identities + Computers + Groups + Policies + Authentication

Active Directory is extremely important for GRC because it is often the **central identity and access-management layer** of a Windows organization.

A GRC professional may assess:

- User access
- Privileged accounts
- Group membership
- Password policies
- Domain administrators
- Dormant accounts
- Joiner/mover/leaver controls
- Access reviews
- Segregation of duties
- Authentication controls

A compromised privileged AD account can potentially affect a large part of the organization.

Group Policy

Group Policy is a Windows mechanism used to **centrally configure and enforce settings for users and computers** in an enterprise environment.

Instead of manually configuring thousands of computers, administrators can define policies centrally and apply them to appropriate users or computers.

Group Policy can control things such as password requirements, account lockout, firewall settings, software restrictions, security configurations, and other Windows behavior.

The basic idea is:

Central Policy → Target Users/Computers → Consistent Configuration

Group Policy is highly relevant to GRC because it helps organizations implement **security policies consistently and demonstrate security configuration controls**.

It supports:

- Security hardening
- Password policies
- Access control
- Endpoint configuration
- Compliance requirements
- Standardization
- Configuration management

Without centralized policy enforcement, organizations can easily end up with inconsistent security configurations.

LDAP

LDAP (Lightweight Directory Access Protocol) is a protocol used to **access and interact with directory services**. A directory stores information about identities and resources, such as users, groups, computers, and organizational information.

LDAP allows applications and systems to query or interact with directory information rather than maintaining completely separate identity databases.

LDAP is a **protocol**, not the same thing as Active Directory. Active Directory supports LDAP, but LDAP can also be used with other directory services.

LDAP is relevant to GRC because centralized directories are important for **identity management and access control**.

Understanding LDAP helps you understand how applications may retrieve:

- User information
- Group membership
- Organizational information

- Authentication-related information

Security is important because improperly configured directory access can expose sensitive identity information.

Kerberos

Kerberos is a network authentication protocol that allows users and systems to prove their identity securely without repeatedly sending their password across the network.

In a Windows Active Directory environment, Kerberos is the primary authentication protocol for domain authentication.

The basic concept is **ticket-based authentication**. After successfully authenticating, a user receives tickets that can be used to access authorized services. This reduces the need to repeatedly provide credentials to every service.

The central component is the **Key Distribution Center (KDC)**, which is typically provided by a Domain Controller in an Active Directory environment.

Kerberos is important for GRC because authentication is a core security control. Understanding it helps you evaluate **identity security, authentication architecture, privileged access, and authentication-related risks**.

It also matters because attacks against enterprise identity infrastructure can have a very large impact.

System Hardening

System hardening is the process of reducing the security weaknesses and unnecessary attack surface of an operating system or system. The goal is to configure the system so that only the functionality, services, users, ports, applications, and privileges that are actually required remain available.

Hardening is not simply "installing antivirus." It involves systematically reducing unnecessary exposure and configuring security controls according to an approved baseline.

Typical hardening activities include disabling unnecessary services, removing unnecessary software, restricting administrative privileges, configuring firewalls, applying security patches, enforcing secure authentication, enabling appropriate logging, and protecting sensitive configurations.

Hardening reduces the number of opportunities an attacker can exploit.

For GRC, this connects directly to:

Security baselines
Risk management
Vulnerability management
Configuration management
Patch management
Access control
Compliance
Audit evidence

A GRC professional may verify whether systems are configured according to an approved security baseline and whether deviations have been identified, approved, and remediated.

Common hardening activities include:

Remove unnecessary software.
Disable unnecessary services.
Close unnecessary ports.
Apply security patches.
Enable host-based firewalls.
Enforce strong authentication.
Restrict administrator privileges.
Apply least privilege.
Enable appropriate logging.
Protect sensitive configuration files.
Disable unnecessary accounts.
Use secure protocols.
Maintain approved configuration baselines.
Regularly assess systems for configuration deviations.